CS-4110 HPC with GPUs

Dr Imran Ashraf
Associate Professor
CS, NUCES-FAST, Islamabad
C-205-E

Objective

- To understand atomic operations
  - Read-modify-write in parallel computation
  - Use of atomic operations in CUDA
  - Why atomic operations reduce memory system throughput
  - How to avoid atomic operations in some parallel algorithms
- Histogramming as an example application of atomic operations
  - Basic histogram algorithm
  - Privatization

A Common Arbitration Pattern

- Multiple customers booking air tickets
- Each
  - Brings up a flight seat map
  - Decides on a seat
  - Update the the seat map, mark the seat as taken

- A bad outcome
  - Multiple passengers ended up booking the same seat

Atomic Operations

thread1:	Old	Mem[x]	thread2:	Old	Mem[x]
	New	Old + 1		New	Old + 1
	Mem[x]	New		Mem[x]	New

If Mem[x] was initially 0, what would the value of Mem[x] be after threads 1 and 2 have completed?

- What does each thread get in their Old variable?

The answer may vary due to data races. To avoid data races, you should use atomic operations

Timing Scenario #1

Time	Thread 1	Thread 2
1	(0) Old Mem[x]	
2	(1) New Old + 1	
3	(1) Mem[x] New	
4		(1) Old Mem[x]
5		(2) New Old + 1
6		(2) Mem[x] New

- Thread 1 Old = 0
- Thread 2 Old = 1
- Mem[x] = 2 after the sequence

Timing Scenario #2

Time	Thread 1	Thread 2
1		(0) Old Mem[x]
2		(1) New Old + 1
3		(1) Mem[x] New
4	(1) Old Mem[x]	
5	(2) New Old + 1	
6	(2) Mem[x] New	

- Thread 1 Old = 1
- Thread 2 Old = 0
- Mem[x] = 2 after the sequence

Timing Scenario #3

Time	Thread 1	Thread 2
1	(0) Old Mem[x]	
2	(1) New Old + 1	
3		(0) Old Mem[x]
4	(1) Mem[x] New	
5		(1) New Old + 1
6		(1) Mem[x] New

- Thread 1 Old = 0
- Thread 2 Old = 0
- Mem[x] = 1 after the sequence

Timing Scenario #4

Time	Thread 1	Thread 2
1		(0) Old Mem[x]
2		(1) New Old + 1
3	(0) Old Mem[x]	
4		(1) Mem[x] New
5	(1) New Old + 1	
6	(1) Mem[x] New	

- Thread 1 Old = 0
- Thread 2 Old = 0
- Mem[x] = 1 after the sequence

Atomic Operations – To Ensure Good Outcomes

thread1: Old Mem[x]
New Old + 1
Mem[x] New

thread2: Old Mem[x]
New Old + 1
Mem[x] New

Or

thread2: Old Mem[x]
New Old + 1
Mem[x] New

thread1: Old Mem[x]
New Old + 1
Mem[x] New

Atomic Operations in General

- Performed by a single ISA instruction on a memory location *address*
  - Read the old value, calculate a new value, and write the new value to the location
- The hardware ensures that no other threads can access the location until the atomic operation is complete
  - Any other threads that access the location will typically be held in a queue until its turn
  - All threads perform the atomic operation **serially**

Atomic Operations in CUDA

- Function calls that are translated into single instructions (a.k.a. *intrinsics*)
  - Atomic add, sub, inc, dec, min, max, exch (exchange), CAS (compare and swap)
  - Read CUDA C programming Guide 4.0 for details

- Atomic Add

  int atomicAdd(int **address**, int **val**);*

  reads the 32-bit word **old** pointed to by **address** in global or shared memory, computes **(old + val)**, and stores the result back to memory at the same address. The function returns **old**.

More Atomic Adds in CUDA

- Unsigned 32-bit integer atomic add
  unsigned int atomicAdd(unsigned int address, unsigned int val);*

- Unsigned 64-bit integer atomic add
  unsigned long long int atomicAdd(unsigned long long int address, unsigned long long int val);*

- Single-precision floating-point atomic add (capability > 2.0)
  - float atomicAdd(float* address, float val);

Histogramming

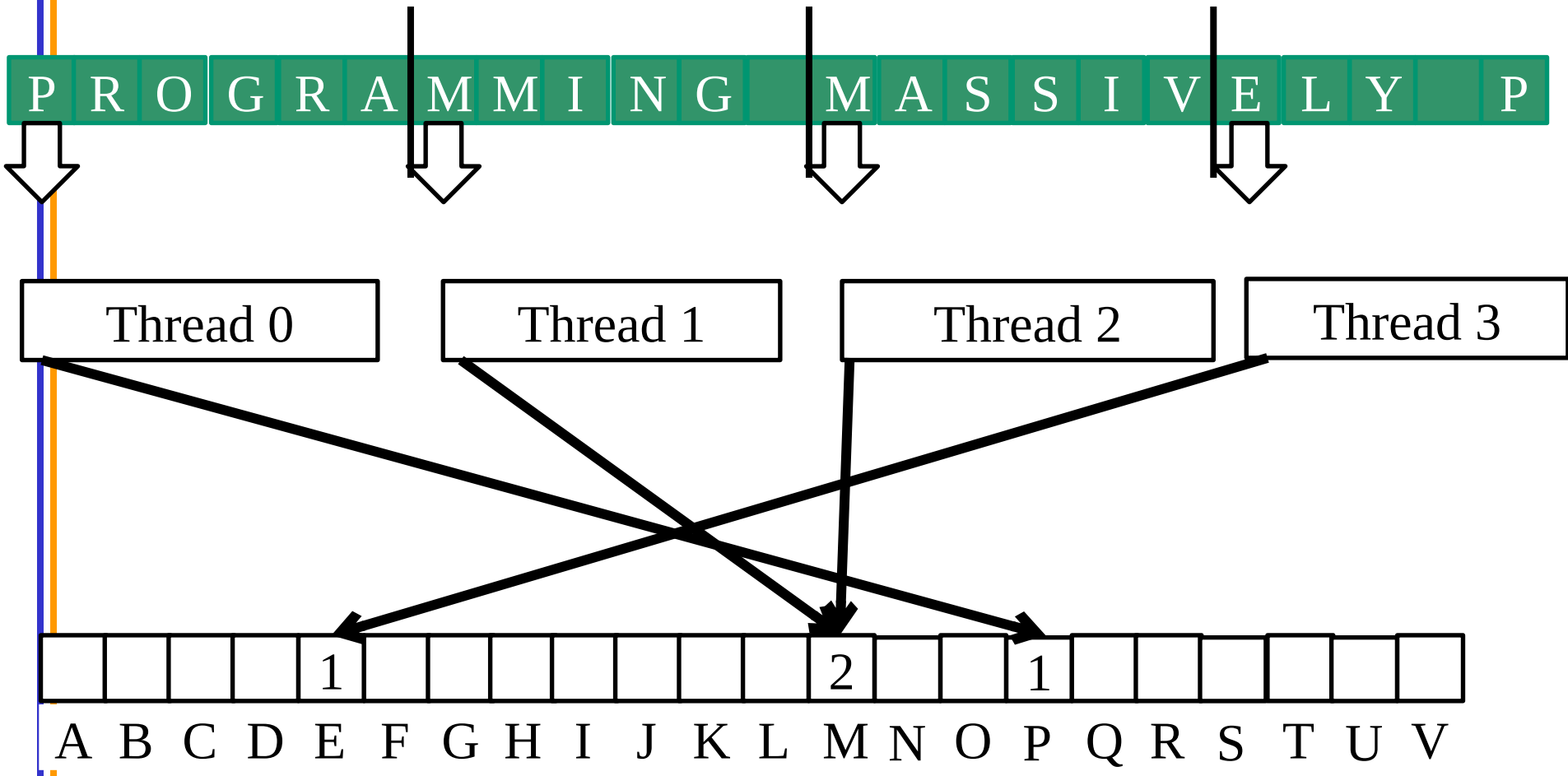
- A method for extracting notable features and patterns from large data sets
  - Feature extraction for object recognition in images
  - Fraud detection in credit card transactions
  - Correlating heavenly object movements in astrophysics
  - …
- Basic histograms - for each element in the data set, use the value to identify a “bin” to increment

A Histogram Example

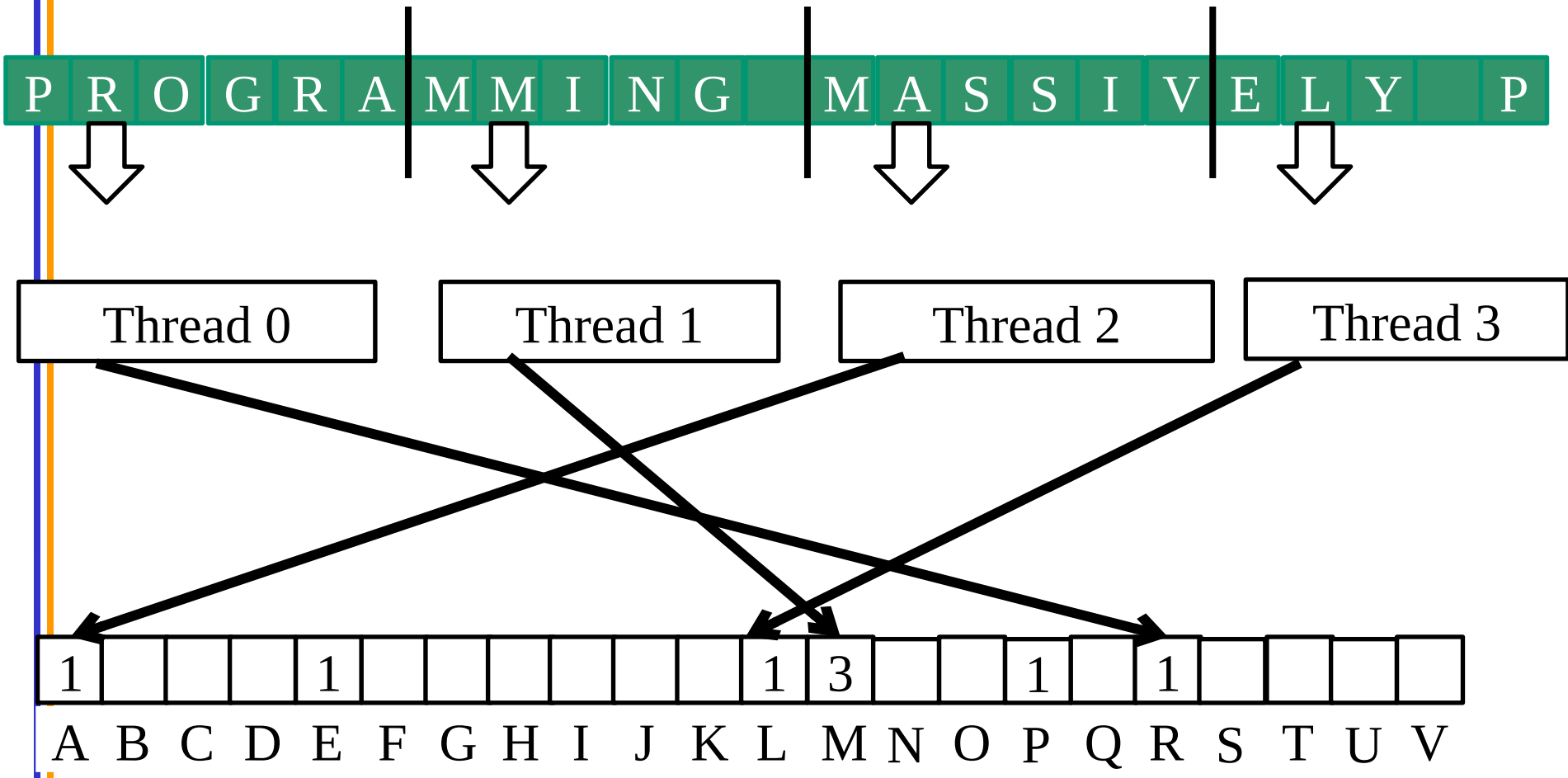
- In sentence “Programming Massively Parallel Processors” build a histogram of frequencies of each letter
- A(4), C(1), E(1), G(1), …

- How do you do this in parallel?

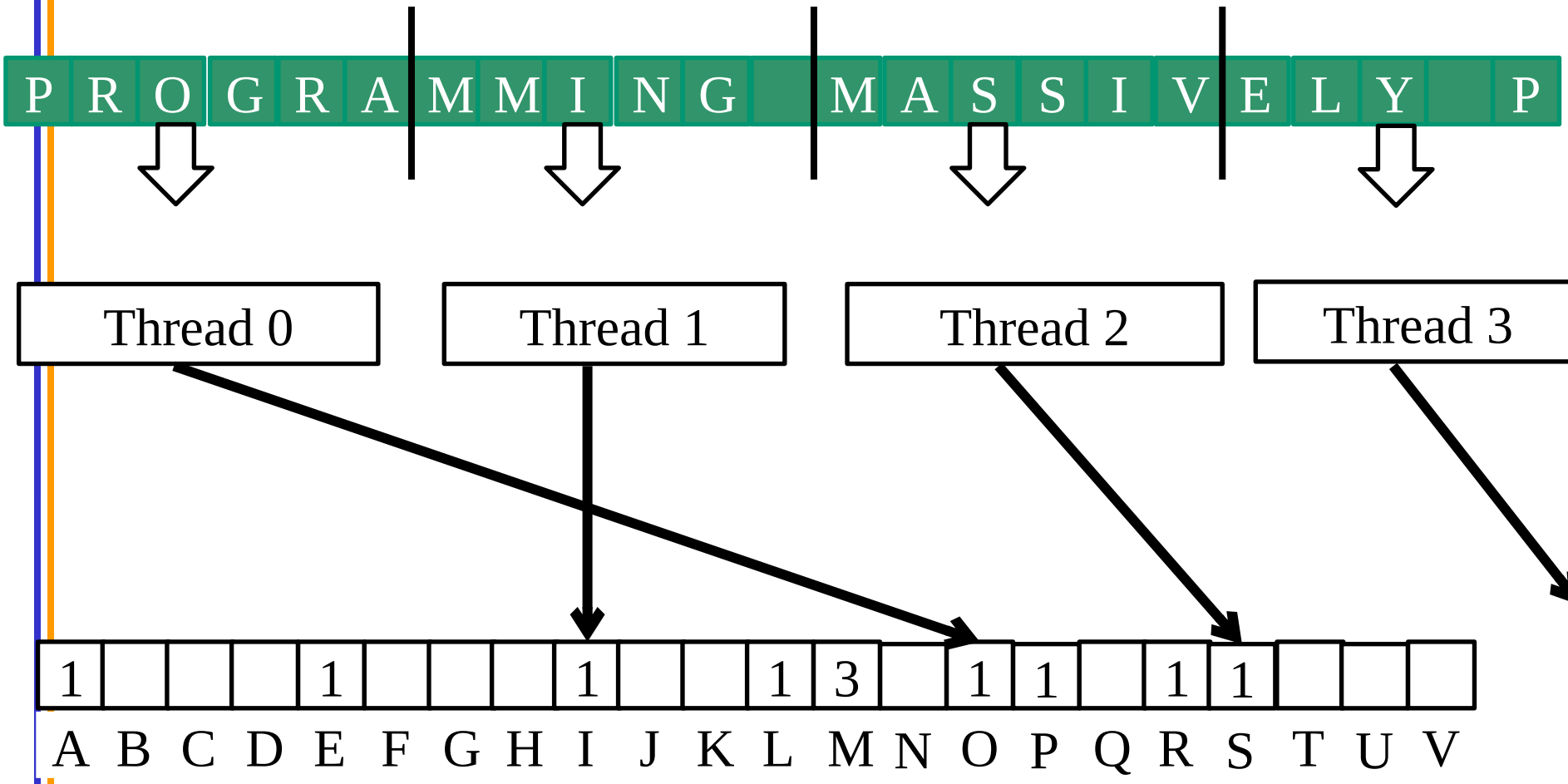
Iteration #1 – 1st letter in each section



Iteration #2 – 2nd letter in each section



Iteration #3


P R O G R A M M I N G M A S S I V E L Y P
Thread 0
Thread 1
Thread 2
Thread 3
1 1 1 1 3 1 1 1 1
A B C D E F G H I J K L M N O P Q R S T U V

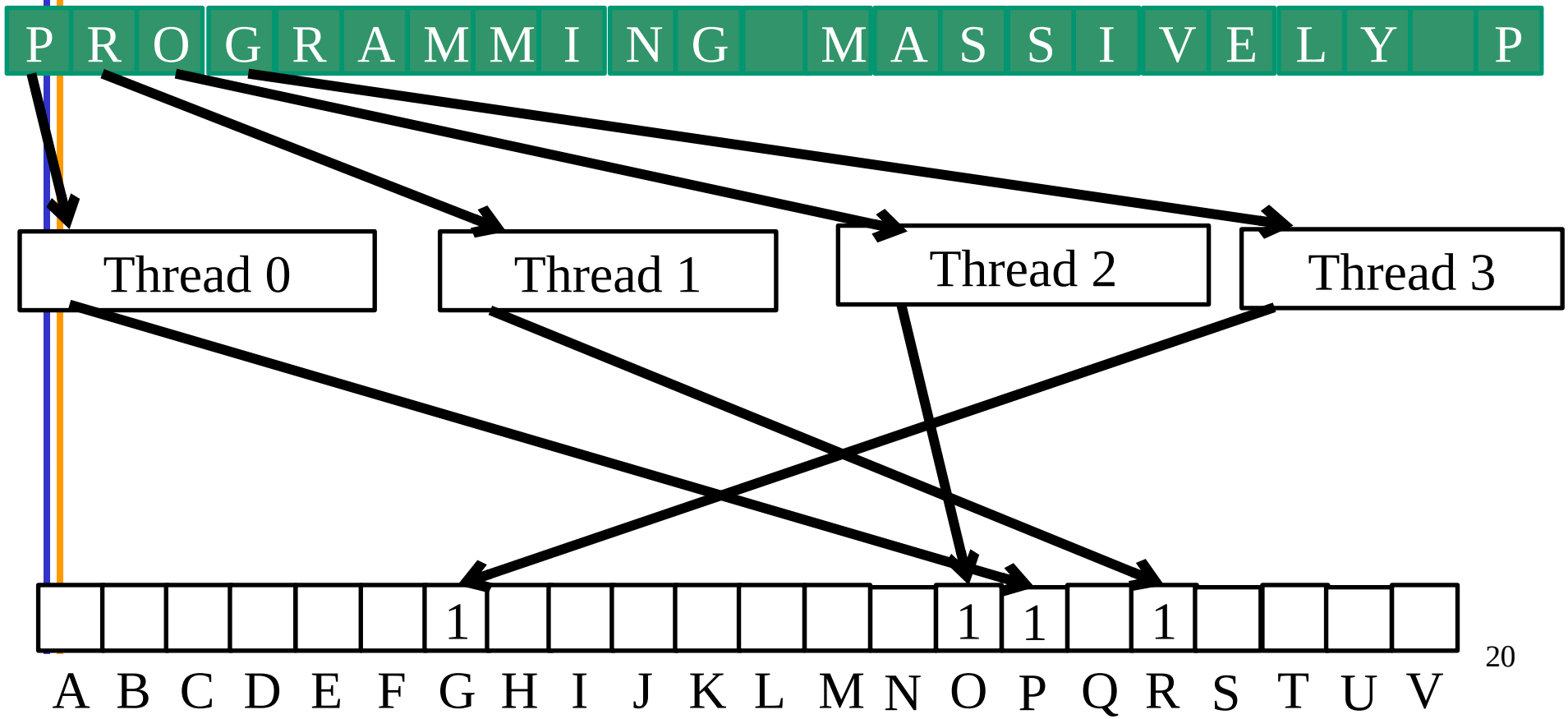
What is wrong with the algorithm?

Objective

- To learn practical histogram programming techniques
  - Basic histogram algorithm using atomic operations
  - Privatization

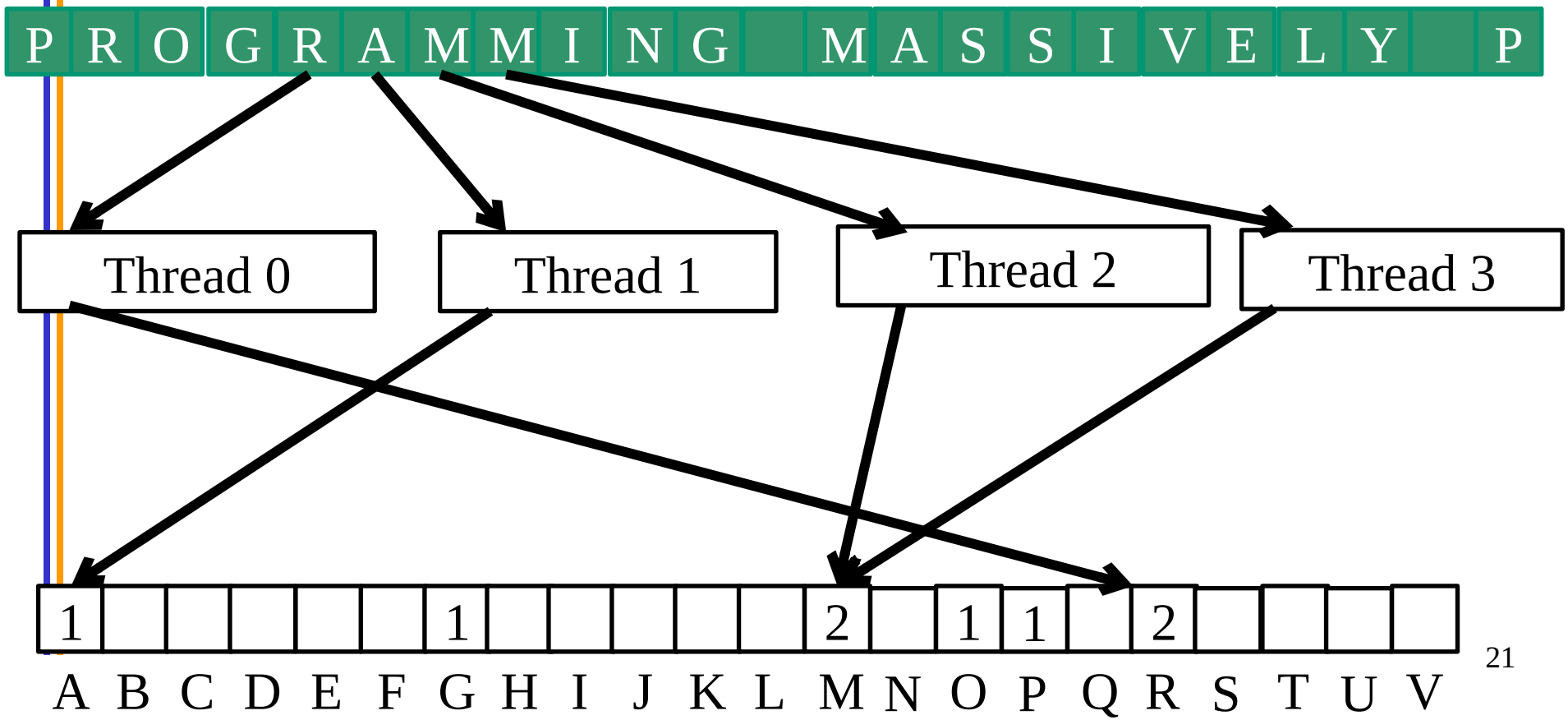
What is wrong with the algorithm?

- Reads from the input array are not coalesced
  - Assign inputs to each thread in a strided pattern
  - Adjacent threads process adjacent input letters



Iteration 2

- All threads move to the next section of input



A Histogram Kernel

- The kernel receives a pointer to the input buffer
- Each thread process the input in a strided pattern

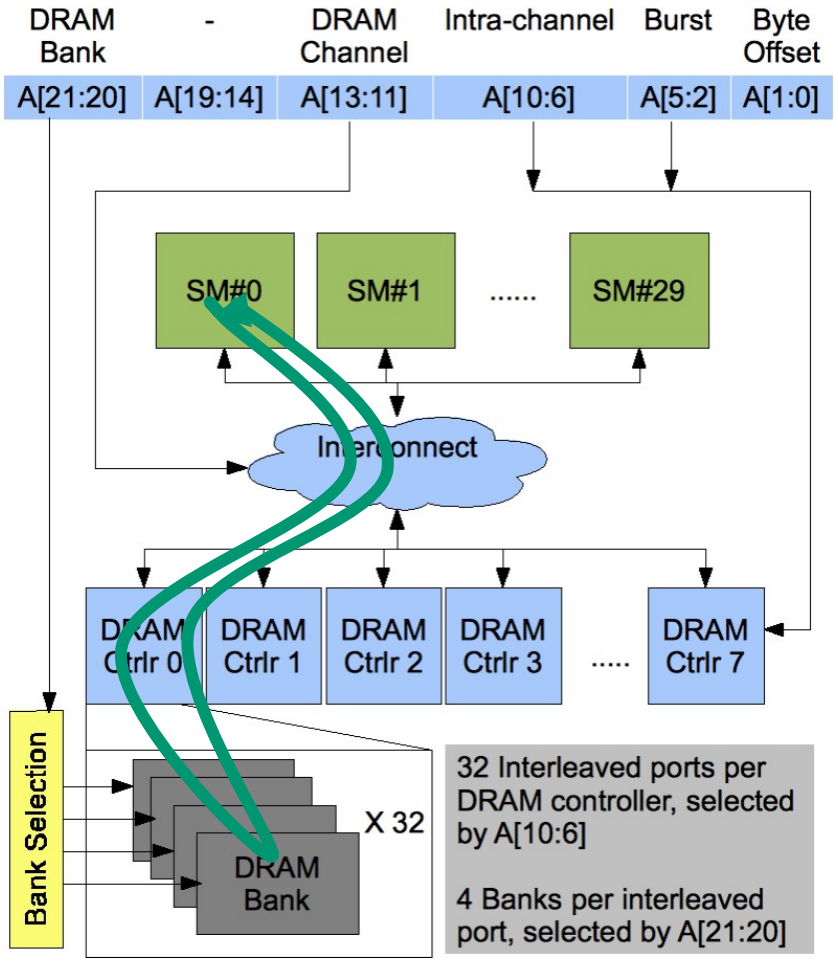
```
__global__ void histo_kernel(unsigned char *buffer,
                             long size, unsigned int *histo)
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;

// stride is total number of threads
    int stride = blockDim.x * gridDim.x;
```

More on the Histogram Kernel

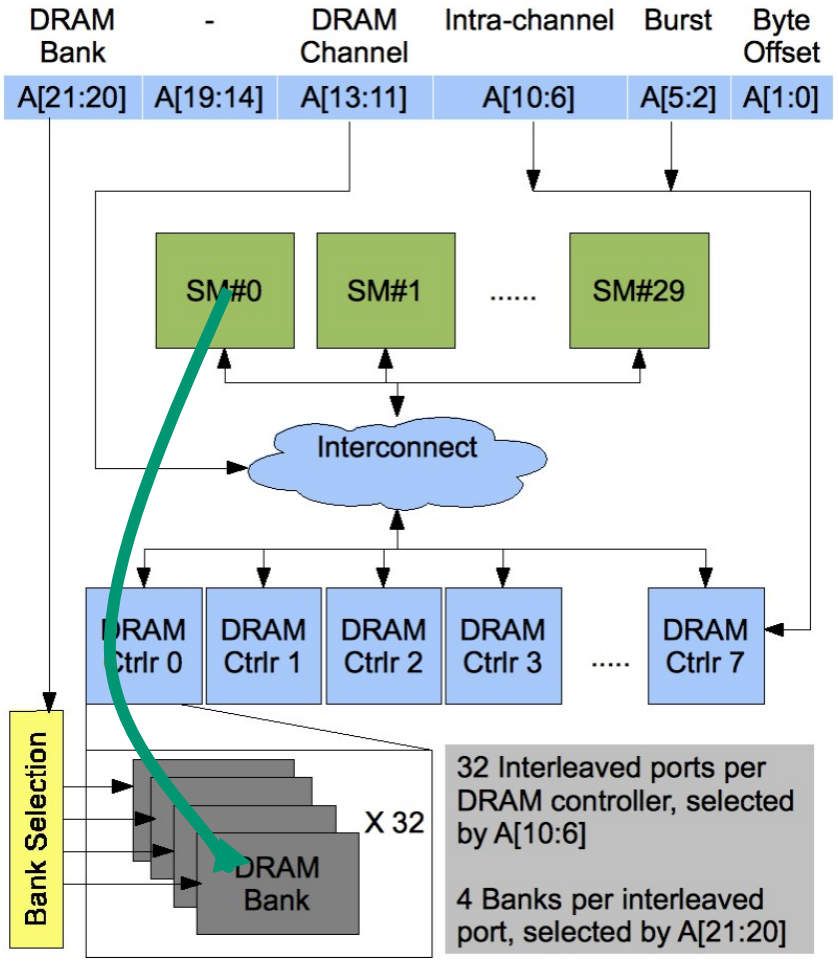
```
  // All threads handle blockDim.x * gridDim.x
  // consecutive elements
  while (i < size) {
      atomicAdd( &(histo[buffer[i]]), 1);
      i += stride;
  }
}
```

Atomic Operations on DRAM



- An atomic operation starts with a read, with a latency of a few hundred cycles

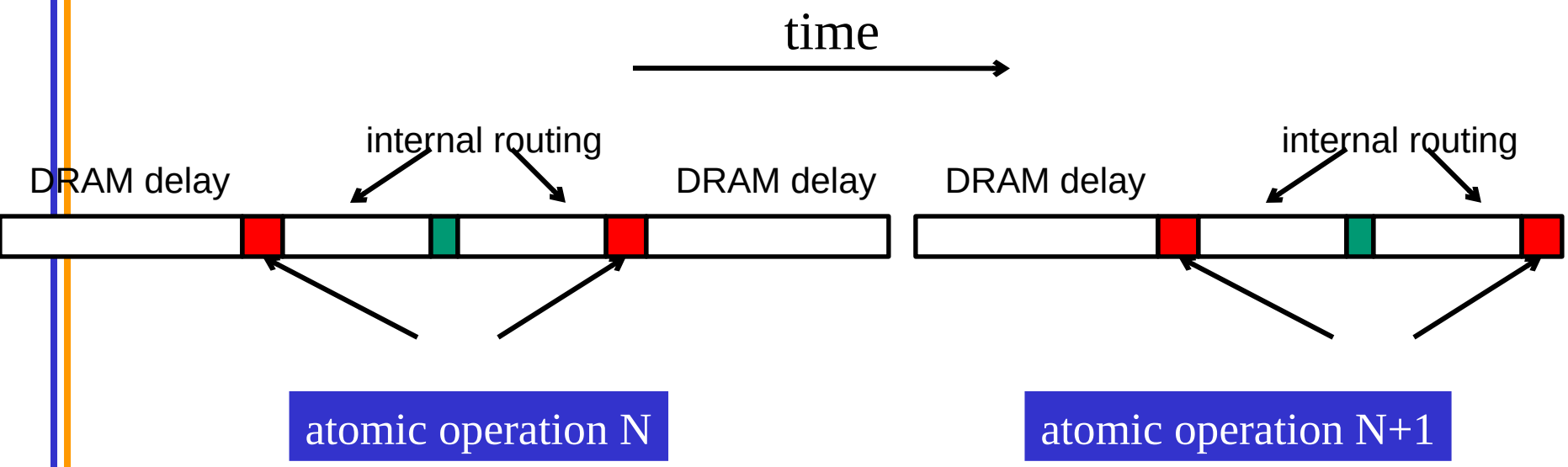
Atomic Operations on DRAM



- An atomic operation starts with a read, with a latency of a few hundred cycles
- The atomic operation ends with a write, with a latency of a few hundred cycles
- During this whole time, no one else can access the location

Atomic Operations on DRAM

- Each Load-Modify-Store has two full memory access delays
  - All atomic operations on the same variable (RAM location) are serialized



Latency determines throughput of atomic operations

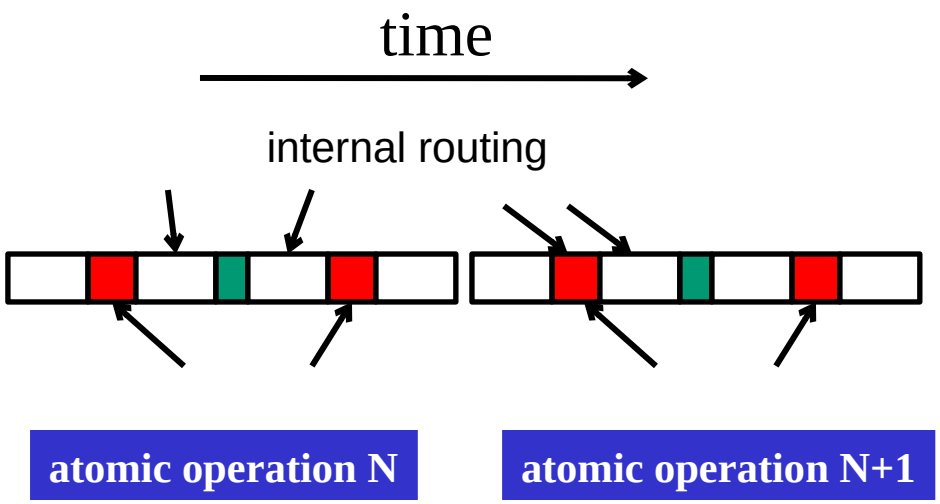
- Throughput of an atomic operation is the rate at which the application can execute an atomic operation on a particular location.
- The rate is limited by the total latency of the read-modify-write sequence, typically more than 1000 cycles for global memory (DRAM) locations.
- This means that if many threads attempt to do atomic operation on the same location (contention), the memory bandwidth is reduced to < 1/1000!

You may have a similar experience in supermarket checkout

- Some customers realize that they missed an item after they started to check out
- They run to the isle and get the item while the line waits
  - The rate of check is reduced due to the long latency of running to the isle and back.
- Imagine a store where every customer starts the check out before they even fetch any of the items
  - The rate of the checkout will be 1 / (entire shopping time of each customer)

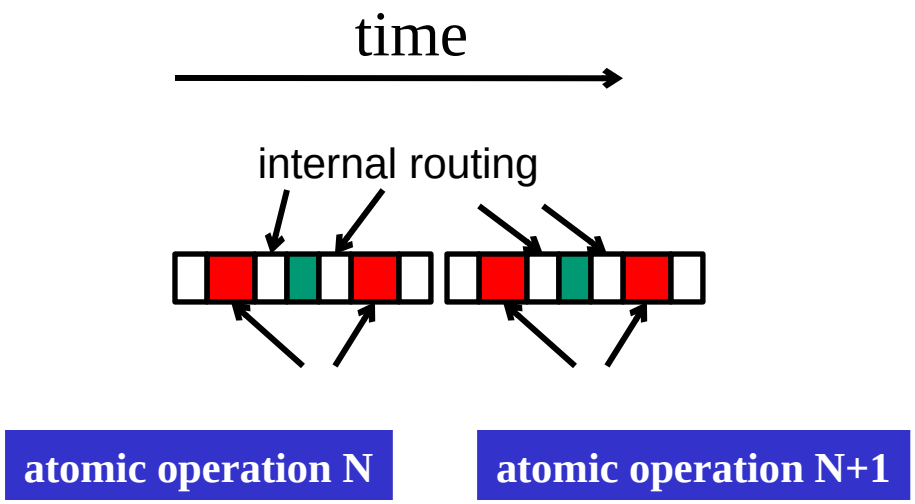
Hardware Improvements (cont.)

- Atomic operations on Fermi L2 cache
  - medium latency, but still serialized
  - Global to all blocks
  - “Free improvement” on Global Memory atomics



Software Improvements

- Atomic operations on Shared Memory
  - Very short latency, but still serialized
  - Private to each thread block
  - Need algorithm work by programmers (more later)



Atomics in Shared Memory Requires Privatization

- Create private copies of the histo[] array for each thread block

```
__global__ void histo_kernel(unsigned char *buffer,
                             long size, unsigned int *histo)
{
   __shared__ unsigned int histo_private[256];
  if (threadIdx.x < 256)
            histo_private[threadidx.x] = 0;

  __syncthreads();
```

Build Private Histogram

```
int i = threadIdx.x + blockIdx.x * blockDim.x;

// stride is total number of threads
int stride = blockDim.x * gridDim.x;

while (i < size) {
    atomicAdd( &(histo_private[buffer[i]), 1);
    i += stride;
}

// wait for all other threads in the block to finish
__syncthreads();
```

Build Final Histogram

```
  if (threadIdx.x < 256)
     atomicAdd( &(histo[threadIdx.x]),
                                  histo_private[threadIdx.x] );

}
```

More on Privatization

- Privatization is a powerful and frequently used techniques for parallelizing applications
- The operation needs to be associative and commutative
  - Histogram add operation is associative and commutative
- The histogram size needs to be small
  - Fits into shared memory
- What if the histogram is too large to privatize?

Other Atomic operations

- atomicCAS (int *p, int cmp, int val)
  - CAS = compare and swap

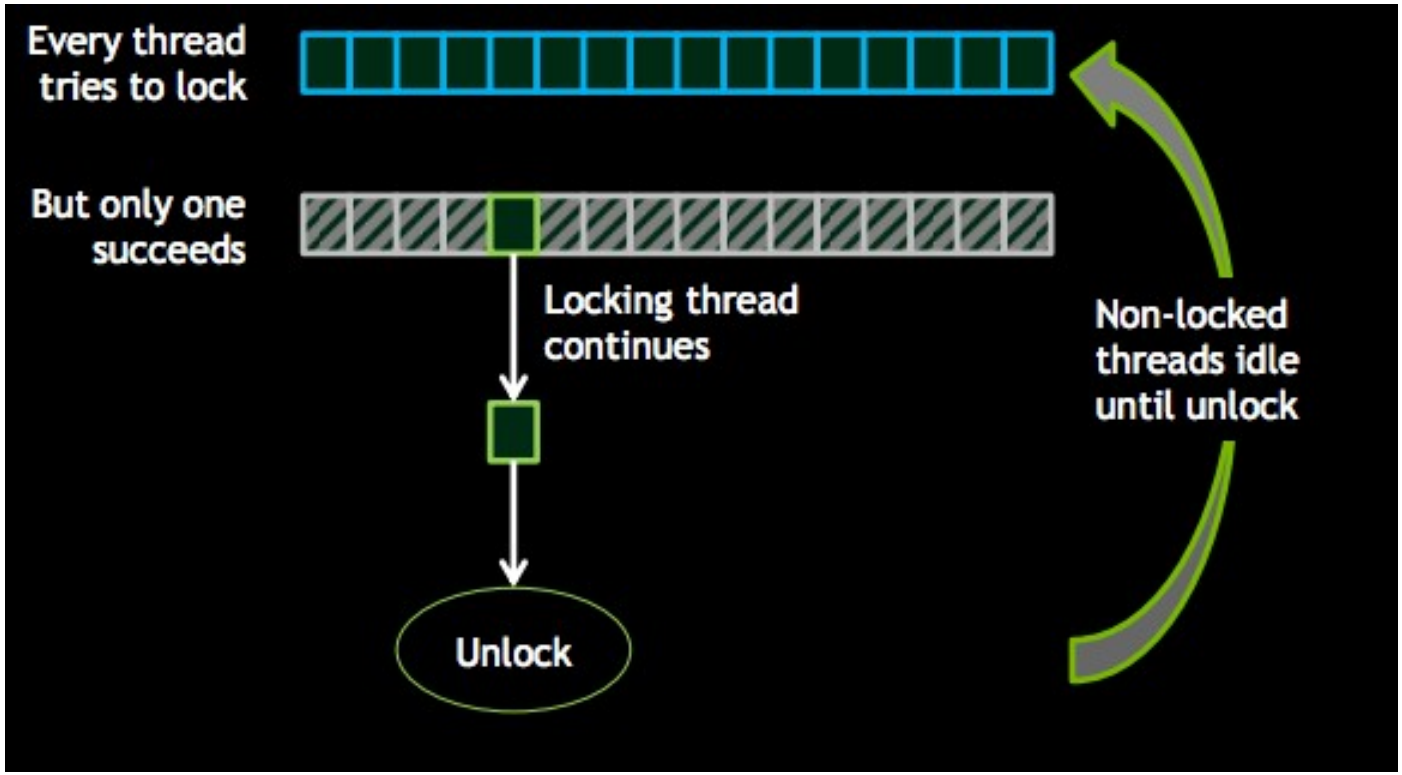
```
//atomically perform the following
int old = *p;
if(cmp == old) *p = v;
return old;
```

- AtomicExch – unconditional version of CAS

```
int old = *p;
*p = v;
return old
```

- What are these used for?

Locking causes control divergence in GPUs



Divergence deadlock if locking thread idles

Alternatives to locking?

- Lock-free algorithms/data structures
  - Update a private copy
  - Try to atomically update a global data structure using compare and swap or similar
  - Retry if failed
  - Need data structures that support this kind of operation

- Wait-free algorithms/data structures
  - Similar to histogramming – don’t wait, but atomic update
  - But applies only to some algorithms

Summary